AZINE

Architecture

Front end, back end, hosting and apps, and why each piece is the way it is

OCTOBER 28, 2026
AZINE.IO · MAIN @ E863F98

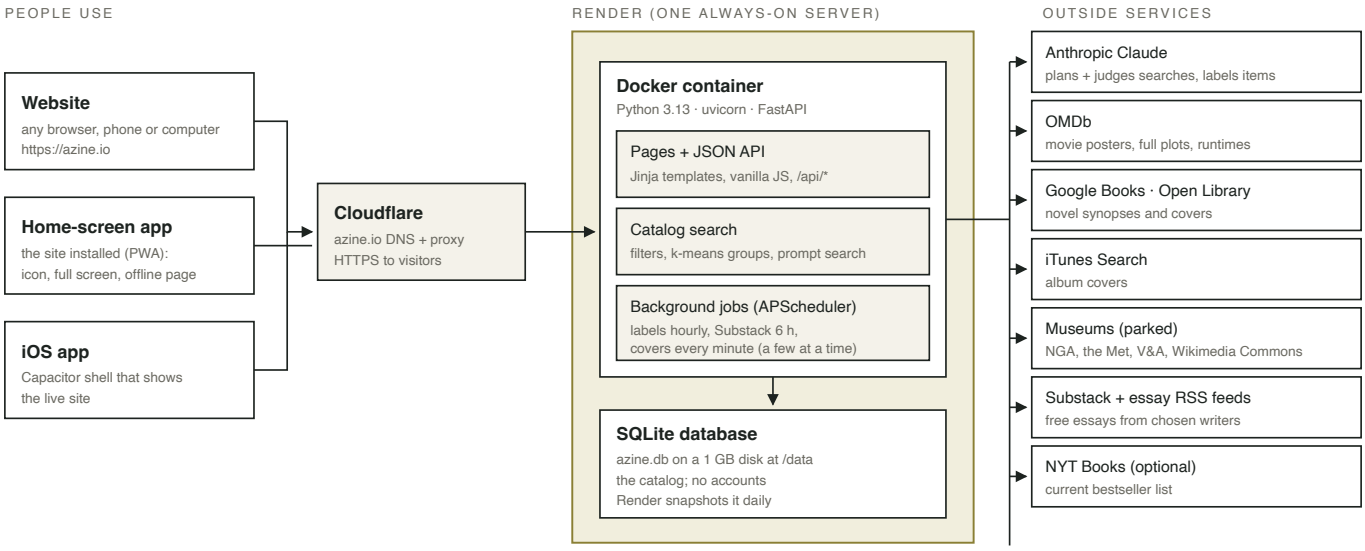
1. What azine is
2. The system at a glance
3. Hosting and deployment
4. Three ways in: website, home-screen app, iOS app
5. Back end: the server
6. Content: catalogs, sources and enrichment
7. Finding things: filters, groups and prompt search
8. Front end: the four letters
9. Operations: limits, costs, backups
10. Decision log
11. Keeping this document current

01 What azine is

azine is a catalog of things worth your time: essays, movies, albums and novels (E-M-A-N), the same for everyone. Every entry is a real work from a curated catalog, mostly pre-2021 (plus festival films from 2022 on) and nothing in it is AI-written. There are no accounts, no personalized picks and no voting. You find things three ways: by category, by when the work was made (and, for movies, albums and novels, by country), and by saying what you're after ("I want to read about x").

> WHY IT'S SHAPED THIS WAY
>
> The product promise is "the best of the internet without the filler." Until Oct 27, 2026 azine delivered it as a personal board built from a taste answer and votes; it is now a catalog anyone can browse, with no sign-up standing between a visitor and the work. Claude is used only behind the scenes: to understand a search prompt, to judge which works fit it, and to label catalog items. It never writes what you read.

02 The system at a glance



Everything runs as one Python server on Render, with its SQLite database on an attached disk. Cloudflare serves the azine.io name; outside services are called only to understand search prompts, label items, and fetch covers and synopses.

What happens when someone searches

1. Cloudflare resolves `azine.io` and passes the request to the Render service over HTTPS.
2. Browsing by category, date or group is a plain database query (`GET /api/catalog`): no Claude, no account.
3. A prompt (`POST /api/search`) goes to Claude Sonnet once to become a search plan, inside the filters already set.
4. The server lines up every work inside the filters, most similar first, then Claude Haiku scores each for how well it fits the whole prompt; everything scoring 75+ comes back, best first, however many that is (~25 s for 1,700 films).
5. Anything shown without a cover is queued, and a background job fetches it within a minute or so.

---

03 Hosting and deployment

PIECE	SETUP
Host	Render web service, Docker runtime, region Oregon, plan 0.5 CPU / 512 MB (~$7/month), always on
Image	`Dockerfile`: `uv` + Python 3.13 slim; dependencies in their own layer; runs `uvicorn azine.main:app --proxy-headers` on `$PORT`
Disk	1 GB persistent disk mounted at `/data`; `AZINE_DB_PATH=/data/azine.db`. It keeps the catalog's fetched covers, synopses and labels across deploys.
Config	`render.yaml` (Blueprint): disk, health check `/health`, env vars. Secrets (`ANTHROPIC_API_KEY`, `OMDB_API_KEY`, `GOOGLE_BOOKS_API_KEY`, `NYT_API_KEY`) are set in the dashboard, never in git.
Domain	`azine.io` registered on Cloudflare: `A @ → 216.24.57.1`, `CNAME www → azine.onrender.com`. Render issues the certificate; `azine.onrender.com` also still works.
Deploys	Every push to `main` builds and deploys. With a disk attached Render stops the old server first, so there's ~1 minute of downtime. Migrations run on start.
Security	No accounts, so no cookies or passwords. uvicorn trusts the proxy's `X-Forwarded-*` headers, which the search rate limit uses to tell visitors apart.

> WHY ONE SERVER, SQLITE AND A DISK
>
> azine's needs are modest: one database file, a few background jobs, and a few hundred requests a day at most. A single always-on process with SQLite on a persistent disk is the simplest setup that does all of that: no separate database server, no job queue, no cluster. The constraints this brings are deliberate: exactly one instance (the SQLite file and the in-process scheduler both assume it), and the disk must exist (the server now refuses to start without it, with a message saying so). The way to grow is a bigger instance; moving to Postgres and a separate worker only becomes necessary if azine ever needs more than one server.

> WHY RENDER (AND NOT THE FREE TIER, FLY.IO, OR VERCEL)
>
> - **Vercel, Netlify, Cloudflare Pages, Heroku** were ruled out: no persistent disk and/or no always-on process, so they'd require Postgres plus an external scheduler.
> - **Fly.io** was set up first (it fits technically), then swapped for **Render** because Render is driven from a web dashboard rather than a command-line tool, and auto-deploys from GitHub.
> - **Render's free tier** sleeps after 15 idle minutes (which would stop the hourly refresh) and can't have a disk.
> - **Docker** keeps it portable: the same image runs on any host that offers a disk at `/data` and HTTPS.

WHY 512 MB IS ENOUGH

Measured peak was ~310 MB with the old board (re-measure for the catalog: each category's embeddings and groups are now held in memory). It used to be ~630 MB: the National Gallery of Art fetch held two ~40 MB CSV slices as text (GitHub gzips them, so an "8 MB" byte range is ~40 MB decoded). It now streams the rows and keeps only the three fields it uses. Memory grows with *simultaneous* activity, not total accounts. Upgrade to 1 CPU / 2 GB if the logs ever show it running out.

04 Three ways in: website, home-screen app, iOS app

Website
Server-rendered pages at azine.io; works in any modern browser, phone or desktop. Every app below is a window onto this.

Home-screen app (PWA)
`manifest.webmanifest`, icons, and a service worker (`/sw.js`). "Add to Home Screen" gives an icon, full-screen launch and an offline page. Nothing to publish.

iOS app
`ios-app/`: Capacitor 8 (Swift Package Manager). Loads `https://azine.io`; bundles only the icon, launch screen and offline page. External links open in Safari / Music / YouTube.

Caching and offline

The service worker is **network-first for everything**: pages and the API are never served from a cache (the board is personal and changes daily), and `/static/` files are cached only as an offline fallback. With no connection, a page load shows `static/offline.html`. The stylesheet's address carries its file's modification time (`style.css?v=…`, from `static_url()`), so every edit is a new URL and browsers can never pair a new page with an old stylesheet.

WHY THE APPS WRAP THE WEBSITE

One codebase serves all three, and a deploy updates every app at once, with no App Store release for content or design changes. The trade-off is App Store review guideline 4.2 ("minimum functionality"), which sometimes rejects pure web wrappers: fine for personal use and TestFlight; for the public store, add something native (e.g. push notifications for new picks). Capacitor 8 uses Swift Package Manager, so the Xcode project was generated without CocoaPods.

WHY NETWORK-FIRST CACHING

Originally static files were served cache-first. After the board redesign, a browser paired the new `board.html` with its cached old `style.css` and showed an unstyled page. Versioned stylesheet URLs plus a network-first service worker make that impossible while keeping the offline page.

05 Back end: the server

Code layout (`src/azine/`)

MODULE	JOB
main.py	Builds the FastAPI app: static files, routers, manifest/service worker routes, `/health`; on start runs migrations, seeds the catalog, starts the scheduler
db.py · models.py	SQLAlchemy engine/session (SQLite), the catalog and museum-art tables
migrations/	Alembic migrations, applied automatically on every start
routers/	`catalog.py` (the API), `pages.py` (pages, robots, sitemap); `art.py` is parked, not mounted

catalog/	`filters.py` (category + date, sorting, acclaim), `embeddings.py`, `groups.py` (k-means groups), `search.py` (prompt search), `enrich.py` (covers and synopses), `details.py` (fetching them in bulk into `data/details/`)
claude.py	The one structured-output request to Claude, shared by search and labeling
content_sources/ · data/	Catalog loaders and live sources (Substack, RSS, museums, OMDb, iTunes, books); curated catalogs and vocabularies
labeling.py · catalog_builder.py	Subject/tone labels (hand-edited JSON + Claude for new items); offline tool that builds the big catalogs from source lists
scheduler.py · seed.py · art_refresh.py	Background jobs; catalog upserts on start; museum image rotation (parked)

HTTP endpoints

ROUTE	PURPOSE
GET /	The front page (`templates/index.html` + `static/js/front.js`): the four letters, then the catalog, all through the API below
GET /about	The owner's letter (public; text in `data/about_letter.txt`)
GET /api/catalog	The catalog filtered by `category` (any of the four, repeatable), `year_from` / `year_to`, `country` (repeatable; essays have none) and optionally a `group`; sorted by acclaim, newest, oldest or a seeded shuffle; paged
GET /api/catalog/groups	Each category's groups, sized to the date and country filters
GET /api/catalog/meta · /{id}	Counts, year ranges and countries (with counts) per category, for building the filter controls; one item
POST /api/search	A prompt plus the same filters: every work Claude judges a 75%+ fit, best first, with the plan it read from the prompt. 30 searches per visitor per hour
/robots.txt · /sitemap.xml · /favicon.ico	For search engines: keep out of /api; list the public pages
/health · /manifest.webmanifest · /sw.js	Health check for Render; the installable-app files

Data model

TABLE	HOLDS
content_items	The catalog: category, title, creator, year, source, url, image, synopsis, tags, subjects, tones, countries, tier, runtime, publish date
art_images	Museum images (parked until the new front end wants them)

Migration 0006 dropped every table that held a reader's data (users, taste signals, recommendations and votes, saved items, palettes, Sunday email issues, outlet connections). The code that used them is at the git tag `personal-recommender-final`.

> WHY MIGRATIONS RUN ON START
>
> The database must never be deleted to pick up a schema change. Alembic migrations are applied automatically when the server starts, locally and on Render, so a deploy that changes the schema upgrades the live database in place.

Background jobs (APScheduler, inside the server process)

JOB	EVERY	DOES

label_new_items	1 hour	Claude labels up to 200 catalog items that have no subjects/tones yet, and up to 600 movies, albums and novels with no countries
refresh_substack	6 hours	Pulls new free posts from the chosen Substack newsletters
fetch_wanted_images	1 min	Fetches covers/synopses for up to 15 items readers were just shown without one
backfill_images	30 min	Fetches 20 more, most acclaimed first, inside the free API quotas

06 Content: catalogs, sources and enrichment

CATEGORY	CATALOG (≈ ITEMS)	WHERE IT COMES FROM
Movies	1,900	Cross-referenced "greatest films" lists, 1940–2021, plus 2022+ films that screened at Cannes, Sundance, Toronto or Hong Kong. Trailers for some; Library of Congress Screening Room switched off.
Novels	1,900	Cross-referenced "greatest novels" lists, 1810 onward; current NYT bestsellers with a key
Albums	1,800	Rolling Stone's 500 Greatest Albums, plus BBC Music Magazine's 31 greatest classical albums (`albums_classical_list.py`), plus ~1,300 from Latin America, South Korea and Africa: winners and nominees of the Latin Grammys and Grammys (album of the year, urban/música urbana, Latin pop, Latin rock, global music), the Korean Music Awards (a critics' jury, each genre's album prize), MAMA and Golden Disc album of the year, the Headies, Rolling Stone Argentina's top 10, and OkayAfrica readers' 100 best African albums. Built by `azine-build-catalog albums` into `albums_extended_list.py`; the Rolling Stone 500 still sorts first by acclaim. Each album is tiered "instrumental" (no words) or "lyrical".
Essays	2,000 + live	About 1,500 picked by people, not algorithms: Longform.org's recommendations (2010–2022; its essay and memoir tags, every "Best Article" and its classics) and the National Magazine Award winners for essays, columns, criticism and feature writing (1977–2025, from Wikipedia); and 60 well-known essays The Atlantic ran before 1931 (Thoreau, Emerson, Douglass, Du Bois, Muir, Zitkala-Ša), now public domain and linked to their free full text on Wikisource or Project Gutenberg (`essays_atlantic_archive.py`, hand-kept). Built by `azine-build-catalog essays` into `essays_extended_list.py`; `list_count` drives the acclaim sort. Plus a hand-edited author list, non-paywalled RSS feeds, and chosen Substack newsletters (free posts only).
Art	live	National Gallery of Art open data, the Met (incl. costume), V&A, Wikimedia Commons museum paintings; Library of Congress paused

Enrichment: fetched in bulk, saved with the code

Covers, synopses, links and runtimes for the whole catalog are fetched on a developer's machine with `uv run azine-fetch-details` (`catalog/details.py`) and saved in `data/details/movie.json`, `album.json` and `novel.json`, one item per line. They are committed like the labels, and seeding applies them on every start, so the live site has them as soon as it's deployed. The command looks up only what's still missing, saves as it goes, and stops after 40 misses in a row (a quota or a block), so it can be rerun the next day. Anything still missing is fetched by the server the old way: an item *shown* without a cover is queued and fetched within a minute or so, and a slower job backfills the rest, most acclaimed first. Either way, `catalog/enrich.py` does the lookup and stores it on the row (pages never wait on these lookups):

- **Movies:** OMDb poster, IMDb link, runtime and the *full* plot (a paragraph, not OMDb's one-liner). A record counts only if its director matches the catalog's (OMDb's title lookup returns namesakes: a podcast episode for Se7en); otherwise OMDb's search is tried, including with "and" spelled "&".

- **Albums:** cover art and the album's Apple Music page from iTunes Search (one lookup gives both). Only a result by the same artist counts (or one with exactly the same, longer title), so an album iTunes doesn't have gets no cover rather than someone else's.
- **Novels:** synopsis from Google Books (publisher descriptions, several sentences; needs a free key), else Open Library. No cover: novels, like essays, show their words (owner's call), so a novel is looked up only while its synopsis is short. Descriptions are cleaned (HTML, markdown, "From Amazon.com", ALL-CAPS banners), cut at a sentence end past ~900 characters, and non-English text is skipped.

A fuller synopsis replaces a catalog one-liner and survives restarts; each item is looked up at most once per server run.

> WHY IN BULK, AND WHY SAVED IN THE REPOSITORY
>
> The trickle (20 every 30 minutes, plus what readers are shown) left most of the live catalog bare: search results came back as plain cards with no synopsis, and what was fetched on one machine never reached the other. Fetching once, ahead of time, and committing the result gives every copy of azine the same covers. Free API quotas are small (OMDb: 1,000 requests/day; Google Books: per-key daily quota; iTunes: ~20 a minute), so the command paces itself and a run may take more than a day.

Labels: subjects and tones

Every item is labeled on two axes from one shared vocabulary (`data/labels_vocab.py`): **subjects** (what it's about) and **tones** (how it feels). Curated labels live in `data/catalog_labels.json` (hand-editable, applied on every start); new items are labeled by Claude hourly. Because the vocabulary is shared, one search reads across categories: "melancholy" means the same on a film and an album. Labels also decide which items can be grouped (below): an item known only by its title has nothing to be grouped by.

Countries (movies, albums, novels; not essays): a movie's production countries, an album's artist's, a novel's author's (an émigré gets both), main one first, at most three, as today's common English names ("Russia" for Soviet work, "Germany" for East or West). Labeled by Claude from what it knows of the work and creator, kept in the same JSON file under `"countries"`.

---

07 Finding things: filters, groups and prompt search

1. Category, date and country (no Claude)

Any of essays, movies, albums and novels, a range of years the work was made (essays use the year published; a full date orders Substack posts within it), and any of the countries. Works with no known year drop out once a date range is set; essays drop out once a country is. Sorted by acclaim (how many source lists name it, scaled within its category so a list count and an album rank compare), newest, oldest or a seeded shuffle that stays put while you page.

2. Embeddings

Each item is a TF-IDF vector blending its labels (genres, subjects, tones; 70% of the weight) with its title and synopsis text (30%). Labels carry most of the signal: measured on the real catalog, 91–94% of same-genre pairs land closer than the nearest 10% of random pairs, versus ~50% for text alone.

3. Groups (no Claude)

`catalog/groups.py` clusters each category with k-means over those embeddings (about one group per 50 items, 6–40 per category) and names each group from the labels its members share far more than the rest of the category ("eerie · mental health · dark"). Only items with a genre tag or a label are grouped: clustered anyway, the title-only items all fell into one meaningless group a third of the catalog wide. They join a group once labeled. Groups are rebuilt when the catalog or its labels change, so group ids are for use within a visit.

4. Prompt search

1. **Plan** (one Sonnet call). Claude reads the prompt, never the catalog, and returns the 1–4 distinct things it asks for ("facets"), each with its own categories, years and countries (only ever narrowing the reader's filters: "funny 90s albums" is albums from 1990–99, "Korean films" is movies from South Korea), subjects and tones from the label vocabulary, keywords, well-known works that typify it, and the groups that fit. "The sea, and also funny 90s albums" is two facets; averaging them would find work that is neither.
2. **Order** (local). Each facet becomes one point in each category's embedding space, and every work inside the filters is lined up nearest first, works Claude named at the front. Facets share the line by how central each is to the prompt, taking turns: the multi-exemplar idea from Pinterest's PinnerSage, with Claude doing the splitting. This only sets the order and what falls past the 2,000 cap.
3. **Judge** (Haiku, batches of 60, 32 at once). Claude scores every work in the line 0–100: it first works out the prompt's essential parts, and a work fits only if all of them hold. Works scoring 63–74 get a second look in a fresh batch and the average of the two. Everything at 75+ comes back, best first, however many that is.

> WHY A JUDGE, AND WHY A 75% BAR INSTEAD OF A TOP 10
>
> Owner's rule: a search returns the works that really fit, not a fixed number. For "a girly film about finding yourself", *Roman Holiday* fits and *Pride and Prejudice* doesn't: the mood matches, but it isn't about self-discovery. Similarity in a TF-IDF space can tell "girly" and "a young woman's story", not that distinction, so it only gathers candidates; Claude, which knows the works, makes the call. Judging only the nearest few hundred missed it: labeled romantic and royal, *Roman Holiday* ranks ~350th by similarity for that prompt, so every work inside the filters is judged. Haiku makes that affordable (1,700 films is a few cents) and, on this prompt, agreed with the owner where Sonnet didn't (Sonnet read *Roman Holiday* as a romance, 62); the plan, one short call that decides which works get named, uses Sonnet, whose plans were far better. Scores near the line vary by a few points between runs (78–85 for *Roman Holiday*), hence the second look. A search's plan is cached, so the same prompt and filters cost one plan.

> WHY TF-IDF
>
> TF-IDF (scikit-learn) separates genres well at a catalog of thousands; a neural embedding model would add a multi-gigabyte dependency for little gain at this size. Worth revisiting if the catalog reaches tens of thousands.

---

08 Front end: the four letters

The old pages (the landing note, sign-up, the board, the quiz, saved items, the weekly page) went with accounts on Oct 27, 2026; they are at the git tag `personal-recommender-final`. The new front page (Oct 28, 2026) is one server-rendered template, `templates/index.html`, plus `static/js/front.js`, which builds everything after the first screen from the catalog API (section 05).

1. **The letters.** E, M, A, N stacked down the left in Helvetica on `#FFF8F0`, and "better for your brain" small and grey in the bottom right.
2. **The title card.** Clicking a letter grows it from where it sits into the big first letter of its title card (**E**SSAYS) while the others fade; the rest of the word and a line about the category (with the catalog's real year range) fade in beside it. The card holds for 3 seconds (a click skips it), then the page becomes that catalog. The address keeps what's open (`azine.io/#movie`, `#movie+album`), so reload and the back button work; "azine" at the top of the filter column (or Esc) goes back to a clean `azine.io`.
3. **The filter column**, on the left: the four letters again (pick one or several), a vertical timeline with the earliest year at the top (drag either end; the years show as `yyyy - yyyy`), and "Vibes", a box for the prompt search (Enter sends it).
4. **The catalog**, in 5 columns, most acclaimed first, 60 at a time as you scroll (`GET /api/catalog`). Movies and albums show only their covers; hovering shows the title, who made it, the year and the synopsis. Essays and novels show

their words: title, author, year, source, synopsis. A cover not yet fetched is a plain card with the title on it.

5. **While a search runs** (`POST /api/search`, ~15–25 s), the middle shows one work of art at a time from `data/landing_wall.py` (28 public-domain works from the Met, stored with the site), with its title, artist, year, a sentence of description and the museum, a new random one every 6 seconds. Then every result, in the same 5 columns. Changing a filter afterwards goes back to the catalog; Enter searches again.

Styles: section 9 of `style.css` and the FRONT PAGE block of its `:root` settings (colors, the letters' font, size and spacing, the zoom time, the title card's sizes and how long it holds, the tagline, the filter column's width, the number of columns). On a phone the letters shrink and the catalog drops to 2 columns.

> WHY NO FRAMEWORK
>
> Plain templates and CSS variables keep every visual decision in one readable file, with nothing to compile and no dependency churn. The page has four states (letters, catalog, waiting, results); a few hundred lines of plain JavaScript cover them.

> WHY ART WHILE SEARCHING
>
> A search takes Claude 15–25 seconds to judge every work in the filters. A spinner makes that feel like waiting; a painting with a line about it makes it part of the visit, and uses images already stored with the site, so it never waits on a museum.

Handwriting font

The board's fixed labels are in the owner's handwriting, as a real font: "Azine Hand" (`static/fonts/azine-hand.woff2`, 23 KB). The owner wrote the alphabet (capitals and lowercase, twice each), punctuation and a heart in a drawing app, and the labels with the digits on a second page. Both were exported as *vector* PDFs, where each pen stroke is a filled outline. `scripts/make_handwriting_font.py` groups the strokes into rows and characters, matches them to the expected letters, sets every letter exactly on the baseline (letters that hang below it, like g and the comma, are placed by the row's fitted line), evens the capitals and digits to one height (`EVEN_HEIGHTS`), merges each character's strokes (skia-pathops) and writes the font (fontTools). The two versions of each letter alternate through an OpenType `calt` rule, so double letters don't repeat exactly. Any element with `class="hand"` uses it: section headings, the details' labels, the date (month written out), the menu, the sort labels and buttons, and "Fetching your recs". Titles, synopses and values stay in the regular fonts. Settings: the HANDWRITING block in `style.css`.

> WHY A FONT, NOT IMAGES OF WORDS
>
> A font handles text that changes (the date) and any future label, stays real text (resizable, recolorable by the CSS settings, readable by screen readers), and is one small file. Cut-out word images would match the page exactly but need re-drawing for every wording change. The vector PDFs made the font lossless: the outlines are the pen strokes themselves, not a trace of pixels.

09 Operations: limits, costs, backups

TOPIC	TODAY
Cost	Render web service ~$7/month + small disk; azine.io yearly at Cloudflare; Claude API usage per search (a Sonnet plan plus Haiku judging every work in the filters: roughly 1–5 cents) and per label batch
Memory	~310 MB peak of 512 MB. Watch Render → Metrics; upgrade to 1 CPU / 2 GB if it nears ~450 MB or restarts for memory
API quotas	OMDb 1,000/day; Google Books per key. The cover jobs stay inside them; searches are capped at 30 per visitor per hour

Backups	Render snapshots the disk daily and keeps them ≥7 days (restoring loses later changes). Own copy: `scp -s` over Render SSH (see DEPLOY.md)
Logs	Render → Logs. Startup refuses to run without the `/data` disk and says so
Deploy time	Build a few minutes; a cold start on an empty database takes ~1 minute (seeding + fetching Substack)
Scaling path	1) bigger instance; 2) only if more than one server is needed: Postgres + a separate worker for the scheduled jobs

10 Decision log

Newest first. Earlier entries (before Sept 30, 2026) predate this document and are reconstructed from the code and its history.

WHEN	DECISION	WHY (AND WHAT ELSE WAS CONSIDERED)
Oct 28	**Public-domain Atlantic essays, linked to free copies**	Owner's request: archival essays from The Atlantic and The New Yorker that aren't paywalled. Both magazines meter every article on their own sites, old ones included, so the free route is the public domain: anything published before 1931. That gives The Atlantic 70 years (from 1857) and The New Yorker only five, mostly humor, almost none of it transcribed online, so it was left out. Sixty well-known Atlantic essays were picked by hand and each link checked to hold the full text: most on Wikisource, many from its transcriptions of the magazine's own issues (which also prove where they ran); the rest in the books they were later collected in, on Project Gutenberg. Marked free, though The Atlantic counts as paywalled. Considered: indexing every Wikisource issue (about 480 pieces, mostly 1857–1866, mixed with poems and fiction, and few by names readers would know).
Oct 28	**Covers and synopses fetched in bulk and committed with the code; movie lookups checked against the director**	Owner's report: a search for movies about friendship and moving returned 'Round Midnight and Dances With Wolves as bare cards, with no cover and no synopsis. The trickle (20 every 30 minutes, plus what readers were shown) had covered only the most acclaimed films on the live site, and the ~750 fetched on the owner's machine never reached it. Now `azine-fetch-details` fetches for the whole catalog and saves the results in `data/details/`, which seeding applies on every start, the same way labels travel. Checking the results showed that OMDb's title lookup returns whatever it finds by that name (a podcast episode for Se7en, a TV documentary for Downfall), so a movie's record now counts only if its director matches, with OMDb's search as a fallback; every film is being looked up again (OMDb's free key allows 1,000 requests a day, so this takes ~3 days), and the checked record replaces a namesake's poster or plot: 32 of the first 460 films had one, Annie Hall and Boyhood among them. Only movies and albums get covers; novels and essays show their words (owner's call), so novels are looked up only for a synopsis. Considered: fetching while the search runs (adds seconds to a 20 s search, and the quota still caps it); keeping the trickle alone (weeks to fill the catalog).
Oct 28	**Movies, albums and novels labeled by country, with a country filter**	Owner's request, alongside the Latin, Korean and African albums: browse by where work comes from. Labeled by Claude (one Sonnet pass over ~5,700 items, a few dollars) rather than from Wikidata: matching every title to Wikidata is slow and error-prone, and Claude knew the countries of the works checked. Today's countries, so a filter for Russia finds Tarkovsky and one for Germany finds Fassbinder. A movie keeps its co-producing countries; an album, its artist's; a novel, its author's. Essays have none (an essay's outlet says little about it), so a country filter leaves them out. Prompt search can narrow by country too ("Korean films"), but only for work *from* there, not set there.
Oct 28	**Albums from Latin America, South Korea and Africa, chosen by award juries**	Owner's request (Bad Bunny, K-pop, Afrobeats): albums were the smallest category and almost entirely Anglo-American. No well-kept all-time critics' list exists for any of the three (Paste's K-pop list is a fan survey and blocks scripts; the Rolling Stone 50 Latin rock list isn't readable), so most come from juried awards on Wikipedia, winners and nominees: ~1,280 albums. A jury's pick is also how a 2022+ album proves it's human-made, as with films. Awards are dated a year after the album, so the year is shifted back. Apple Music's first search result was sometimes someone else's album (a Grinch soundtrack for Da Grin), so cover lookups now insist on the same artist. OkayAfrica's readers' list has no years (93 albums), so those drop out of date-filtered views until a year is found.

Oct 28	**Front page redesign: E M A N stacked down the left; a click grows the letter into a title card**	Owner's design, drawn in Canva. The four corners and hover text gave way to the letters in one column, and the line about each category moved from the hover onto a title card (ESSAYS, the E large), shown for 3 seconds before the catalog (owner's call; considered: keeping the card above the catalog, or waiting for a click). The zoom now lands on the card's letter, not past the edges of the screen. Letters in Helvetica rather than IBM Plex Mono, to match the design. Added "better for your brain", small and grey, on the letters screen only.
Oct 28	**The new front page: E, N, M, A in the corners; zoom into one for its catalog**	Owner's design, tried first as a clickable mockup. The letters are the only thing on the first screen; hovering explains each, which is how a visitor picks. Once a catalog is open the other letters move into the filter column, so categories can be combined. The timeline is vertical (earliest at the top) beside the works rather than a slider above them. Movies and albums show only covers (synopsis on hover), because the cover is how people recognize them; essays and novels have no cover worth showing, so they show their words. The Met wall from the old landing page fills the 15–25 s search with one work at a time; each work got a one-line description in `landing_wall.py`. Letters in IBM Plex Mono for now, not the handwriting font (owner's call). Considered: a spinner while searching; keeping the letters in the corners once a catalog is open (they crowded the columns).
Oct 28	**Essays from the Longform archive and the National Magazine Awards**	Essays were the thinnest category (~500, mostly recent feed posts) and had no acclaim signal. Both new sources are chosen by editors or juries, which keeps the catalog AI-free: Longform stopped in December 2022, and NMA winners are picked by a jury. Only the parts of Longform's ~13,000 picks that read as essays are taken (the essay and memoir tags, Best Articles, classics), not all its reported journalism, so essays don't swamp the other three. NMA's Profile Writing table names only magazines, so it's left out. Considered: Best American Essays (no single table of contents to read), public-domain classics (later).
Oct 27	**Search judges every work in the filters; Sonnet plans, Haiku judges; whole catalog labeled and ~750 movie synopses fetched**	Tested on the owner's example ("a girly film about finding yourself", 1950–2016): Roman Holiday never came back. It had no labels or synopsis, the plan didn't name it, and once labeled it still ranked ~350th by similarity, past the 150–300 that were judged. Now all 1,668 films are judged (Haiku, ~25 s, a few cents), near-misses get a second look, and each part of a prompt carries its own categories and years. The bar then moved from 80 to 75 (owner's call: works like Breakfast at Tiffany's sat at 78–82 and dropped in and out between runs). Haiku's plans were poor (it named Pride and Prejudice, invented a "watch" category), so the single plan call uses Sonnet; as judge, Haiku agreed with the owner on Roman Holiday where Sonnet didn't, at ~1/15 the cost. Labeled the 2,400 unlabeled items, and fetched OMDb plots and posters for 753 movies (the old code fetched them only for picked movies).
Oct 27	**azine becomes a catalog: no accounts, no personal picks, no votes; filter by category, date and prompt**	Owner's direction: a collection of AI-free E-M-A-N anyone can browse, rather than a personal recommender. Category and date filters and k-means groups need no Claude. A prompt goes to Claude Haiku twice: once to plan (facets, labels, works, groups; it can narrow the reader's filters, never widen them) and once to judge candidates gathered locally; every work scoring 80%+ comes back, not a fixed number, because similarity alone can't tell "girly" from "about finding yourself". Migration 0006 drops all reader data (3 accounts); the code is at tag `personal-recommender-final`. Parked: museum art, the iOS app, the Sunday email (it needs addresses, i.e. sign-ups). Considered: no Claude at all for search (free, but word matching misses "sad" vs "melancholy"); sending Claude the catalog (too many tokens per search).

Oct 15	**The Sunday email is sent from azine's Google mailbox, not Resend**	Owner's preference: a letter sent by azine itself, not through a service. Sending straight from the server isn't realistic (mail from an unknown cloud server is mostly refused or marked spam), so it goes through azine's existing Google mailbox with an app password: readers see that address and their replies reach the owner's inbox. Render blocks outgoing mail only on its free plan. Resend (a sending service) was the first choice: better at reaching inboxes in bulk, but another account and invisible to readers anyway. Google's limit (2,000/day on Workspace) is far above azine's size.
Oct 15	**Trying the Sunday email on the owner alone**	Owner's request: test real emails before readers get them. `AZINE_NEWSLETTER_ONLY` limits sending (the Sunday job and the command) to the listed addresses and the server refuses any other; Run on the live site (Render Shell) so the email's private links work from a phone; a local send would link to localhost.
Oct 14	**The Sunday email in sections: essays, novels, albums, movies**	Owner's layout: a week-of-the-year heading, essays as linked titles (text only), the novel with its synopsis, albums and movies with their cover and synopsis; replaces a grid of covers. Synopses are cut at a sentence end around 320 characters in the email (the week's page shows them whole). Tapping a title or cover opens the week's page at that pick.
Oct 14	**A Sunday email, opening a private weekly page**	Owner's idea: less friction -- the picks arrive instead of waiting to be visited. One email per reader with their own picks, Sunday 6pm in their time zone; sorting and saving on a phone-sized page reached by a signed link, no login, while saved items and taste stay on the site. Considered: voting links inside the email (every tap opens a browser page anyway, and corporate mail scanners "click" links, which would cast false votes); texting the link like Partiful (higher open rates, but needs phone numbers, consent and US carrier registration -- kept for later, the page works for both); one shared issue on Substack (loses the personalization). Every word is the owner's (`newsletter_copy.txt`); nothing sends while any is blank. On by default, off in Settings or one tap from the email.
Oct 9	**A third vote: "neutral"**	Owner's call: some picks are neither loved nor disliked. Neutral means "not this one" without saying anything about the kind of thing, so it only stops that pick coming back. Considered and turned down: a faint like (lots of "meh" would pull the board toward safe middle picks) and a faint dislike (would push away things that were fine). No database change: votes are stored as text.
Oct 9	**Taste answer: 20 words minimum (was 50)**	Owner's call: 50 words asked a lot of a first-time visitor before they'd seen anything. Claude still gets enough to start a board from 20, and "Refine" and the first week's votes fill in the rest.
Oct 8	**Albums open in Apple Music or Spotify**	Owner's request: albums had no link at all. A page can't detect installed apps (no permission exists for it), so tapping an album offers both. Apple Music gets the exact album from the iTunes lookup already used for covers; Spotify gets a search, since an exact lookup would need a Spotify developer key.
Oct 8	**Landing note fits a phone; its arrow is drawn live**	On a phone the whole 30-square sheet was squeezed to the width: the writing was small and ran under "about / log in". Now the writing alone fills the width, below the header. The arrow was part of the photo, so it stopped well short of the box on tall screens; drawn live, it always reaches it. Considered: a separate phone image (the same crop is done in CSS from the one image).
Oct 8	**Handwriting font: letters set exactly on the line**	Owner's report: the letters' bottoms sat on slightly different lines. As written they wandered by up to a sixth of their height and capitals ranged ~600–950 units tall; each letter now sits on the baseline and capitals/digits are evened to one height. The letters keep their shapes; text runs about 7% wider.

Oct 2	**Landing opens on a fixed wall of Met art; About page; SEO basics**	Owner's design: art first, the note second, the same art for every visitor. Lighthouse SEO was 91/100 (no page description) and the only public page's words were in an image; the About page adds readable text, plus description, link previews, robots.txt and sitemap. Also: the Met retired its /v1/search API (410 Gone), which had quietly stopped one of the board's art sources; moved to /v1.1/search.
Oct 1	**Board labels in the owner's handwriting, as a font**	The owner's hand fits azine's "made by a person" point. A font (vs. cut-out word images) handles the date and future labels and stays real text. Built from vector PDFs, so lossless. New wording from the owner's sheet: "Fetching your recs", "Loved it ♥", "Not for me :(", the month written out.
Oct 1	**Vibes page removed from the board**	Owner's call: fun, but not needed. The wall now leads straight into Read. The palette is still computed, because it colors the stand-in cards for picks with no cover; the mood words and the `daily_vibes` table stay (unused) so the page could return without a migration.
Oct 1	**Classical albums: BBC Music Magazine's 31; an "instrumental only" setting**	Rolling Stone's list had no classical. Classical is recommended to people who say they like it, who don't much like music, or who don't want words (they get only wordless albums, enforced in the recommender, not left to Claude). Blurbs are short factual notes (performers, label, year), not the magazine's reviews. Fixed tiers along the way: Trans Europe Express and Another Green World have vocals, and Homogenic / The Diary of Alicia Keys had been tagged classical.
Oct 1	**Essay search runs in parallel with a 12-second budget**	The first live sign-up stuck on the last painting: the live essay search ran 20 outlets × every liked topic one at a time, up to 5 s each, which could take minutes. Now 12 searches run at once and whatever has come back after 12 s is used (8 topics: ~10 s). Also speeds up every user's daily essay refresh.
Oct 1	**Landing page is the handwritten note; Claude reads the answer during sign-up**	The note says why azine exists in the owner's own hand. Reading the answer in the background (instead of after sign-up) hides most of Claude's time behind the sign-up form. Jobs live in memory, fine for one server; /quiz is the fallback if one is lost.
Oct 1	**Four paintings as the loading bar; essays built last**	Discrete stages read better than a spinner. Essays take ~20 s (live search), the rest ~2 s, so essays go last: three paintings fill quickly and the wait sits on the final one. Images: Monet, Derain, Krøyer are public domain; Hopper's *High Noon* (1949) is public domain in the US per Wikimedia Commons (published without a copyright notice) but may be protected elsewhere. Derain's *View of the Thames* (1906) is the image the owner supplied.
Oct 1	**Domain azine.io on Cloudflare**	azine.com is registered (since 1998) and parked for sale on Sedo; azine.io was free. Cloudflare records must be "DNS only" while Render verifies and issues the certificate.
Oct 1	**Refuse to start without the /data disk**	A service created without its disk crashed with SQLite's unhelpful "unable to open database file"; running without the disk would silently lose every account on each deploy.
Oct 1	**Versioned stylesheet URL + network-first service worker**	A cached old stylesheet with the new board showed an unstyled page. Considered: cache-busting by hand. Chosen: automatic, from the file's modification time.
Oct 1	**Movies use OMDb's full plot**	Multi-sentence synopses read better in the showcase's main column; one-time upgrade for movies already picked.
Oct 1	**Novel synopses + covers from Google Books / Open Library**	90% of novels had no synopsis or cover. Google Books has the fuller descriptions but needs a key (keyless quota is shared and exhausted); Open Library is the fallback and the cover source. Claude-written synopses were considered and not used (no AI-written content).

Oct 1	**Showcases: synopsis right, title left; essays as translucent text cards; no borders; "better for your brain" replaces the number**	Owner's design direction after seeing the first version.
Oct 1	**Wall stays discrete; the rest scrolls normally; showcases after huyml.co**	Discrete screens made content vanish on scroll. Unused wall images now fade in place instead of floating up. Scroll-snapping tried and removed.
Oct 1	**Stream the NGA art CSVs**	The fetch used ~400 MB to pick 2 images (gzip ranges decode ~5× larger). Server peak 630 → 310 MB, fitting Render's 512 MB size.
Oct 1	**Host on Render (replacing Fly.io)**	Dashboard-driven, auto-deploys from GitHub. Free tier ruled out; Vercel/Netlify/Heroku ruled out (no disk / no always-on process).
Oct 1	**App icon is the "A"; launch screen keeps the wordmark**	The "A" became the logo in the header; a square mark reads better as an icon.
Sept 30	**iOS app as a Capacitor wrapper; installable web app**	One codebase, updates without App Store releases. Native rewrite considered: far more work for the same content. Risk: App Store guideline 4.2.
Sept 30	**Docker image + configurable database path**	Portable across hosts; the database lives on a disk that survives deploys.
Sept 30	**Saved items and daily vibes (migration 0003)**	Saved copies outlive rotating boxes; vibes are computed once per board instead of on every load.
Sept 29	**Shared subject/tone labels + taste profile (migration 0002)**	Lets a taste read in one category carry into the others; Claude's guesses weighted below explicit choices.
Sept 28	**SQLite + Alembic migrations applied on start (baseline 0001)**	Never delete the database to change the schema; migrations upgrade it in place everywhere.
Earlier	**Free-text taste prompt read by Claude, list quiz as fallback**	One natural answer replaces a long quiz, and captures dislikes too. Claude only chooses from the real catalog.
Earlier	**PinnerSage-style taste clusters over TF-IDF embeddings**	Preserves distinct tastes instead of averaging them; TF-IDF separates genres well without a multi-GB model.
Earlier	**Enrich items when picked, not in bulk**	Stays inside free API quotas.
Earlier	**Server-rendered pages, vanilla JS, CSS variables**	Hand-tunable look, nothing to build.

11 Keeping this document current

The sources live in `docs/architecture/` (`architecture.html`, `summary.html`, `system.svg`, `print.css`); the PDFs are `docs/azine-architecture.pdf` and `docs/azine-summary.pdf`. After any significant change (hosting, data model, a new

feature or flow, a major design change, a new outside service) update the affected sections, add a row to the decision log, rebuild with `node docs/architecture/build.js`, and commit the PDFs alongside the change. `CLAUDE.md` asks Claude to do this and to share the updated PDFs.